

Actividad 3 - Estructura de datos

Estudiante: Víctor Andrés Hernández Castillo

AL07139940

Profesor: Edgar Iván Patricio Aizpuru

Explicación técnica:

Explica con tus propias palabras:

- **¿Cuál es el caso base del desafío 1?**

Ocurre cuando la variable clave llega a 0, momento en el cual la función detiene las llamadas y simplemente retorna 0

- **¿Cuál es el caso recursivo?**

El problema se divide obteniendo el último dígito con el operador módulo (clave % 10) y reduciendo el número original al dividirlo entre 10 (clave / 10). Luego, se llama a sí misma sumando los dígitos recursivamente y, si la suma resultante es mayor a 9, ejecuta una segunda llamada recursiva para seguir sumando hasta que quede un solo dígito.

- **¿Cómo se aplica divide y vencerás en el desafío 2?**

El algoritmo toma el arreglo completo de señales y calcula la posición central ($\text{mitad} = (\text{inicio} + \text{fin}) / 2$) para partir el problema en dos subarreglos. Esto se repite recursivamente procesando la mitad izquierda y luego la derecha hasta llegar al caso base, que es cuando el subarreglo tiene un solo elemento ($\text{inicio} == \text{fin}$).

- **¿Cómo se divide el problema?**

El algoritmo parte el arreglo en 2 subarreglos llamados izquierda y derecha, para luego usar la función con ambos arreglos para que se dividan sucesivamente hasta que solo quede 1 elemento en cada lado.

- **¿Cómo se combinan los resultados?**

A medida que las llamadas recursivas terminan, los resultados de la mitad izquierda y derecha se evalúan utilizando `Math.max(izquierda, derecha)`. Esto sube por el árbol de recursión comparando los valores más altos de cada mitad hasta devolver el número máximo de todo el arreglo.

- **¿Cómo funciona el backtracking del desafío 3?**

El algoritmo explora sistemáticamente las cuatro direcciones posibles (arriba, abajo, izquierda, derecha) avanzando celda por celda. Si llega a un callejón

sin salida, hace un paso atrás y elimina la última coordenada de la lista camino, restaurando el valor original de la celda en el laberinto para intentar una ruta alternativa.

- **¿Cómo evitas ciclos?**

Antes de moverse, verifica si la próxima celda es una pared (#) o si ya fue visitada (V). Al entrar a una celda válida, la marca temporalmente con una V para evitar volver a procesarla durante el mismo trayecto.

- **¿Cómo determinas cuál ruta es la más corta?**

Si logra llegar a la meta ($r == tr \ \&\& \ c == tc$), el camino actual se guarda directamente como el nuevo mejor camino, ya que el programa se encarga de eliminar automáticamente caminos que no sean mas cortos que el mejor camino.

- **¿Qué optimización aplicaste y por qué?**

En lugar de buscar todos los caminos posibles hasta el final y luego compararlos, el código detiene la exploración si la longitud del camino actual iguala o supera el tamaño del mejor camino encontrado previamente, de esa forma ahorra un tremendo poder de procesamiento al descartar de inmediato cualquier ruta que matemáticamente ya no pueda ser la más corta.

Pruebas:

Entrada de la parte 1:

```
        return contrasena;
    }
    public static void main(String[] args) {
        System.out.println("La primera parte de la contraseña es: " + getContraseña(84729)+getContraseña(56318)+getContraseña(92746));
    }
}
```

Entrada de la parte 2:

```
public static void main(String[] args) {
    int[] señales = {418, 732, 156, 894, 327, 641, 285, 519,763, 204, 947, 386, 675, 128, 856, 493};
    System.out.println("La segunda parte de la contraseña es: " + getContraseña(señales, 0, señales.length - 1));
}
```

Salida:

```
Problems  Output  Debug Console  Terminal  Ports
PS D:\Actividades_estructuras_de_datos\Actividadesitas> java Recursividad.java
La primera parte de la contraseña es: 351
PS D:\Actividades_estructuras_de_datos\Actividadesitas> java Divide_y_venceras.java
La segunda parte de la contraseña es: 947
PS D:\Actividades_estructuras_de_datos\Actividadesitas> |
```

Parte 3 (clave del laberinto: #3XW6PU):

LABERINTO #3XW6PU • 12x12 • SEED: 3XW6PU

LABERINTO A RESOLVER

TU CÓDIGO JAVA (BACKTRACKING)

```
150 }
151 }
152
153 // ===== (YA ESTA HECHO: lectura del laberinto) =====
154 static List<String> leerLaberinto() throws IOException {
155     List<String> lineas = new ArrayList<>();
156     File f = new File("/str/maze.txt");
157     Scanner sc = null;
158     try {
159         if (f.exists()) sc = new Scanner(f);
160     } catch (Exception e) {
161         sc = null;
162     }
163     // Si no hay archivo (en tu IDE), lee de la entrada estandar
164     if (sc == null) sc = new Scanner(System.in);
165     while (sc.hasNextLine()) {
166         String l = sc.nextLine().trim();
167         if (!l.isEmpty()) lineas.add(l);
168     }
169     sc.close();
170     return lineas;
171 }
172 }
173
```

Este es el laberinto para tu código. ¿Dudas de dónde va? [ver instrucciones](#)

Formato de salida: imprime cada posición visitada como fila,columna (una por línea) y al final el camino elegido como RUTA:r0,c0;r1,c1;... (todo en UNA sola línea).

Entrada: el laberinto se lee automáticamente (archivo maze.txt en el navegador, o por System.in en tu IDE). Usa la plantilla como guía. [¿cómo trabajar en local \(IDE\)?](#)

EJECUTAR **Nuevo laberinto** **Volver a dificultad** **Salir**

ESCAPE COMPLETADO Laberinto: #3XW6PU **CAMINO MÁS CORTO**

18 MOVIMIENTOS	80980 RETROCESOS	81053 ESTADOS EXPLORADOS	18 ÓPTIMO (BFS)
--------------------------	----------------------------	------------------------------------	---------------------------

Reflexión final:

1. **¿Qué diferencia existe entre una solución recursiva y una solución iterativa?**
Una solución iterativa repite un bloque de instrucciones mediante ciclos lógicos (for, while) y gestiona sus variables localmente. Una solución recursiva resuelve el problema llamándose a sí misma para procesar versiones cada vez más pequeñas de los datos.
2. **¿Por qué divide y vencerás puede facilitar la resolución de ciertos problemas?**
Debido a que divide y vencerás permite abordar problemas masivos fragmentándolos en subproblemas idénticos y manejables. Al resolver las partes de manera aislada y combinar los resultados, se simplifica el código y frecuentemente se reduce la complejidad.
3. **¿Por qué encontrar una solución con backtracking no significa necesariamente que sea la mejor solución?**

Debido a que la naturaleza del backtracking estándar es explorar ramas hasta encontrar la primera configuración válida, pero esa primera respuesta exitosa no siempre es la más eficiente.

4. ¿Qué problema podría presentarse si el algoritmo no controla las posiciones ya visitadas?

El algoritmo avanzaría a una celda adyacente y luego regresaría inmediatamente a la anterior, rebotando en un bucle cerrado hasta agotar la memoria y provocar un error del sistema.

5. ¿Qué aprendiste al tener que encontrar la ruta más corta en lugar de simplemente encontrar una ruta?

Al buscar obtener la ruta más corta aprendí como implementar un estado global para guardar récords, aplicar lógica matemática para descartar rutas ineficientes en tiempo real y optimizar exhaustivamente el código para evitar que el procesamiento de miles de ramificaciones colapse el sistema.